AEE. Bitácora III. Conexiones Empresariales
Taller Técnico: Operación Escudo

Fase Práctica: Hardening y Vigilancia en la Trinchera

3. Fase Práctica: *Hardening* y Vigilancia en la Trinchera

Reto A: Auditoría de Accesos e Intrusiones (Log Analysis y Scripting)

El archivo de registro de autenticación es vuestra "caja negra". Si alguien intenta forzar la entrada utilizando técnicas de *Fuerza Bruta* o *Ataques de Diccionario*, el rastro indeleble quedará grabado aquí.

1. **Generación de ataques controlados**. Desde vuestro equipo host (Windows) o desde una segunda máquina virtual, realiza al menos 5 intentos fallidos de conexión SSH contra vuestro servidor (ej: ejecutando *ssh administrador_falso@ip_del_servidor* y metiendo contraseñas inventadas).
2. **Análisis Forense Manual**. Accede a */var/log/auth.log* utilizando herramientas de visualización rápida como *tail -n 50*, *less* o filtrando con *grep*. Debéis ser capaces de identificar y hacer una captura de pantalla donde se vea claramente: la dirección IP de origen del atacante, el puerto efímero utilizado y el mensaje literal de error del sistema (algo parecido a *Failed password for invalid user...*).
3. **Automatización inteligente con Bash**. Como imaginaréis, un administrador no puede pasarse la mañana leyendo miles de líneas de texto plano. Debéis crear un *script* en *bash* en la ruta *scripts/check_intruders.sh* que automatice esta tarea. El script debe:
    - Filtrar exclusivamente los intentos fallidos de conexión de las últimas horas. *Pista*: Herramientas como *grep*, *awk* o *sed* serán vuestras mejores amigas para extraer columnas de texto.
    - Contar matemáticamente el número total de errores detectados.
    - Generar (o sobrescribir) un archivo de reporte limpio llamado **alertas.txt** que incluya un encabezado con la fecha y hora exacta de la generación del informe.
    - Requisito de calidad. El script no puede ser un bloque de código incomprensible. Debe estar **debidamente documentado** mediante **comentarios (#)** explicando qué hace exactamente cada variable, tubería (|) y comando utilizado.

Reto B: Configuración del "Escudo" Perimetral con UFW

Bajo el capó de Ubuntu trabaja el potente motor de filtrado de paquetes *iptables* (o *nftables*). Sin embargo, nosotros utilizaremos **UFW (Uncomplicated Firewall)**, una interfaz amigable que nos facilita la vida. Vamos a aplicar la arquitectura más segura: una política de "Denegación por Defecto". Todo lo que no esté explícitamente invitado a entrar, se quedará fuera.

1. **Reglas de Sigilo** (*Drop* vs *Reject*). Configura el firewall para que el servidor ignore completamente las peticiones ICMP Echo Request (el famoso comando ping). Un servidor que hace drop (dejar caer el paquete sin responder) a los pings, se vuelve prácticamente invisible frente a los escaneos de red superficiales que buscan hosts vivos.
2. **Reglas de Servicio**. Por otro lado, si bloqueamos todo, nosotros tampoco podremos administrarlo. Permitid el acceso TCP exclusivamente al puerto 22 (SSH). Si, como buena práctica, habéis cambiado el puerto SSH por defecto a uno superior, aseguraos de abrir ese puerto específico.
3. **Activación y Evidencias**. Ejecuta *sudo ufw enable*. Posteriormente, verifica el estado y la carga real de las reglas con los comandos *sudo ufw status numbered* y *sudo ufw status verbose*.
4. **Prueba cruzada**. En vuestra documentación, debéis incluir capturas demostrando que desde la máquina anfitriona el *ping* da error de tiempo de espera, pero la conexión por cliente SSH (como Windows PowerShell o el Terminal) sí se establece correctamente.

AEE. Bitácora III. Conexiones Empresariales
Taller Técnico: Operación Escudo

Fase Práctica: Hardening y Vigilancia en la Trinchera

Reto A: Auditoría de Accesos e Intrusiones (Log Analysis y Scripting)

El archivo de registro de autenticación es vuestra "caja negra". Si alguien intenta forzar la entrada utilizando técnicas de Fuerza Bruta o Ataques de Diccionario, el rastro indeleble quedará grabado aquí.

1. Generación de ataques controlados. Desde vuestro equipo host (Windows) o desde una segunda máquina virtual, realiza al menos 5 intentos fallidos de conexión SSH contra vuestro servidor (ej: ejecutando ssh administrador_falso@ip_del_servidor y metiendo contraseñas inventadas).

2. Análisis Forense Manual. Accede a /var/log/auth.log utilizando herramientas de visualización rápida como tail -n 50, less o filtrando con grep. Debéis ser capaces de identificar y hacer una captura de pantalla donde se vea claramente: la dirección IP de origen del atacante, el puerto efímero utilizado y el mensaje literal de error del sistema (algo parecido a *Failed password for invalid user...*).

3. Automatización inteligente con Bash. Como imaginaréis, un administrador no puede pasarse la mañana leyendo miles de líneas de texto plano. Debéis crear un script en bash en la ruta scripts/check_intruders.sh que automatice esta tarea. El script debe:

- Filtrar exclusivamente los intentos fallidos de conexión de las últimas horas. *Pista: Herramientas como grep, awk o sed serán vuestras mejores amigas para extraer columnas de texto*.
- Contar matemáticamente el número total de errores detectados.
- Generar (o sobrescribir) un archivo de reporte limpio llamado alertas.txt que incluya un encabezado con la fecha y hora exacta de la generación del informe.
- **Requisito de calidad:** El script no puede ser un bloque de código incomprensible. Debe estar debidamente documentado mediante comentarios (#) explicando qué hace exactamente cada variable, tubería (|) y comando utilizado.

Resolución Paso a Paso del Reto A: Auditoría de Accesos

1. Localización y Diagnóstico Inicial

> Lo primero es saber dónde estamos. En sistemas basados en Debian/Ubuntu, el historial de autenticación se guarda en /var/log/auth.log.

¿Por qué no puede leerlo cualquier usuario?

> ***Respuesta**:* Porque contiene información sensible. Si un usuario invitado pudiera leerlo, vería quién intenta entrar, desde qué IPs y, lo que es peor, podría ver contraseñas que alguien escribió por error en el campo de "usuario". La verdad es que es una mina de oro para el reconocimiento de un atacante.

2. Identificación de Patrones

Para encontrar los fallos de acceso, necesitamos buscar una cadena de texto específica: "Failed password".

Comando rápido para inspeccionar:

sudo grep "Failed password" /var/log/auth.log | tail -n 20

```
pepegil@SIServer-PepeGil:~$ sudo grep "Failed password" /var/log/auth.log | tail -n 20
[sudo] password for pepegil:
2026-04-17T12:14:38.480558+00:00 SIServer-PepeGil sudo:  pepegil : TTY=tty1 ; PWD=/home/pepegil ; USER=root ; COMMAND=/u
uth.log
pepegil@SIServer-PepeGil:~$
```

3. Creación del Script de Automatización

Hacer el grep a mano está bien para una vez, pero un administrador pro automatiza. Vamos a crear un script que haga el trabajo sucio por nosotros.

Paso A: Crear el archivo

mkdir -p ~/scripts

```
pepegil@SIServer-PepeGil:~$ mkdir -p ~/scripts
```

nano ~/scripts/check_intruders.sh

```
pepegil@SIServer-PepeGil:~$ nano ~/scripts/check_intruders.sh
```

Paso B: El código del script

```
  GNU nano 7.2                                                    /home/pepegil/scr
LOG FILE="/var/log/auth.log"
OUTPUT_FILE="$HOME/alertas.txt"

ECHO "--- INFORME DE SEGURIDAD - $(date) ---"> $OUTPUT_FILE
ECHO "Detectando intentos fallidos de acceso" >> $OUTPUT_FILE
```

```
  GNU nano 7.2                                    /home/pepegil/scripts/check_intruders.sh
LOG FILE="/var/log/auth.log"
OUTPUT_FILE="$HOME/alertas.txt"

ECHO "--- INFORME DE SEGURIDAD - $(date) ---"> $OUTPUT_FILE
ECHO "Detectando intentos fallidos de acceso" >> $OUTPUT_FILE

sudo grep "Failed password" $LOG_FILE | awk '{print $ (NF-3)}' | sort | uniq -c | sort -nr >> $OUTPUT_FILE

echo "Infrome generado con exito en %OUTPUT_FILE"s
```

Paso C: Permisos de ejecución

Si no le damos permisos, el sistema dirá que "no".

chmod +x ~/scripts/check_intruders.sh

```
pepegil@SIServer-PepeGil:~$ chmod + ~/scripts/check_intruders.sh
```

4. Ejecución y Verificación

Ahora, ejecuto mi shell:

./scripts/check_intruders.sh

```
pepegil@SIServer-PepeGil:~$ ./scripts/check_intruders.sh
```

Y compruebo el resultado:

cat ~/alertas.txtclear

```
pepegil@SIServer-PepeGil:~$ cat~/alertas.txt
```

Reto B: Configuración del "Escudo" Perimetral con UFW

Bajo el capó de Ubuntu trabaja el potente motor de filtrado de paquetes iptables (o nftables). Sin embargo, nosotros utilizaremos UFW (Uncomplicated Firewall), una interfaz amigable que nos facilita la vida.

Vamos a aplicar la arquitectura más segura: una política de "Denegación por Defecto". Todo lo que no esté explícitamente invitado a entrar, se quedará fuera.

1. Reglas de Sigilo (Drop vs Reject). Configura el firewall para que el servidor ignore completamente las peticiones ICMP Echo Request (el famoso comando ping). Un servidor que hace *drop* (dejar caer el paquete sin responder) a los pings, se vuelve prácticamente invisible frente a los escaneos de red superficiales que buscan hosts vivos.Ç

sudo nano /etc/ufw/before.rules

icmp-type echo-request

2. Reglas de Servicio. Por otro lado, si bloqueamos todo, nosotros tampoco podremos administrarlo. Permitid el acceso TCP exclusivamente al puerto 22 (SSH). Si, como buena práctica, habéis cambiado el puerto SSH por defecto a uno superior, aseguraos de abrir ese puerto específico.

sudo ufw default deny incoming

3. Activación y Evidencias. Ejecuta sudo ufw enable. Posteriormente, verifica el estado y la carga real de las reglas con los comandos sudo ufw status numbered y sudo ufw status verbose.

sudo ufw enable

4. Prueba cruzada. En vuestra documentación, debéis incluir capturas demostrando que desde la máquina anfitriona el ping da error de tiempo de espera, pero la conexión por cliente SSH (como Windows PowerShell o el Terminal) sí se establece correctamente.

ping [192.168.20.183/22]

ssh pepegil@[192.168.20.183/22]